

# Hunting Mr. X: Wellington Edition

Ming Bao

**Abstract**—University Open Days need demonstrations that prospective students can try for themselves, while hidden-movement board games need a group of friends around one table. This project turns such a game into an online multiplayer game, played over the internet in a web browser on a real map, as a showpiece for Open Days. *Hunting Mr. X: Wellington Edition* is a real-time multiplayer game in which one player, Mr X, hides on a 216-node, 372-edge graph of Wellington’s bus, train, ferry and e-scooter network while up to five detectives hunt for him. The engineering challenge is keeping several browsers consistent through server push while guaranteeing that the hidden position never reaches a detective’s device in an unintended fashion, across a Java server and a TypeScript client. The system uses an authoritative Spring Boot server holding all rules in plain Java, per-player filtered views pushed over STOMP and WebSocket, and a Vue and MapLibre GL client that draws a pre-computed graph over open map tiles. In evaluation, 135 automated tests reached 99.2% line coverage and killed all 199 generated mutants, and seven players gave a System Usability Scale score of 72.5, above the average of 68. The game is deployed and was demonstrated successfully at a university Open Day.

**Index Terms**—Multiplayer game, hidden information, server push, WebSocket, map overlay, online multiplayer, Open Day.

## I. INTRODUCTION

**H**IDDEN-MOVEMENT games are board games in which one player moves in secret and the others try to work out where they are. In the best-known example, Scotland Yard by Ravensburger [1], one player (Mr X) travels across a city by taxi, bus and underground while detectives chase him. His position is shown only on a few fixed rounds; in between, the detectives see only which kind of ticket he spent. The rules are simple, but a physical game has two limits: everyone must be in the same room, and the hidden player must record his moves honestly on paper.

An online version removes both. A server can hold Mr X’s position, check every move against the rules, and show each player only what they are allowed to see. Players can then join from their own device, from anywhere, with a link or code.

### A. Problem

The problem is deliberately narrow: *build a browser-based, real-time multiplayer hidden-movement game, played on a map of a real city, in which the rules and the secret are enforced by the system rather than by the players.* Wellington was chosen because a familiar map makes the game instantly recognisable to local players and Open Day visitors.

The engineering difficulty lies in two places:

- *Server push to many clients.* Every move must reach all other players quickly, over busy networks, without any client being trusted with state it should not see. Every

update is therefore several different messages, one per viewer.

- *Separate frontend and backend languages.* The backend is written in Java and the frontend in TypeScript, so they cannot share code. Both sides must agree on the format of every request and message sent between them.

### B. Two Themes

Two themes run through the project and shape decisions in every later section.

*A showpiece for Open Days.* The university’s Open Days need demonstrations that show prospective students what engineering students build, in a form a visitor can try in a few minutes. A game on a map of their own city, with no app to install or account to create, fits that purpose: several groups can play at once on the stand’s laptops while a big screen shows the board.

*Online multiplayer over the internet.* The board game can only be played by people sitting at the same table. Here each player joins from their own computer, wherever they are, using a link or a six-character code, and the server keeps every player’s screen up to date as moves happen. This is what turns the board game into a networked system, and it is where most of the engineering effort lies.

Sustainability is a secondary consideration. Every transport mode in the game is a low-emission one (e-scooter, bus, train and ferry, with the board game’s taxi replaced by the e-scooter), and the infrastructure was kept small; both points are discussed in Section IV.

### C. Goals

The goals were set in the proposal and refined into measurable targets in Section III; Section VI returns to each:

- 1) Enforce every game rule on the server: movement, tickets, turn order, reveal rounds, the double move and Invisible ticket, and every end condition.
- 2) Never send Mr X’s position to a detective’s browser except on a reveal round.
- 3) Reach a System Usability Scale (SUS) score above the average of 68.

### D. Contributions and Findings

The project delivers a deployed, playable game [2]; a curated, pre-computed graph of the Wellington region together with the tool used to build it; an authoritative server that filters state per viewer; and an evaluation combining a map-library benchmark, a layered automated test suite checked by coverage and mutation testing, and user testing with a usability questionnaire. All three goals were met. The main limitations

are a small user sample and the lack of automated frontend tests.

The mechanics are used with Ravensburger's permission for this non-commercial academic project. The original name is not used, so the game is called *Hunting Mr. X*, and the board game's taxi is replaced by the e-scooter, the short-range, low-emission mode a Wellington player is most likely to use today.

Section II reviews related games and technologies, Section III sets out requirements and scope, Sections IV and V describe the design and its implementation, Section VI evaluates the result, and Sections VII and VIII discuss and conclude.

## II. RELATED WORK

### A. Map-Based and Location-Based Games

Games that use real geography fall into two groups. *Map-based* games use a real map as the board while the player sits still. GeoGuessr [3] drops players into a street-level panorama and asks them to place it on a world map; its popularity shows that recognising real places is enjoyable in itself. But GeoGuessr's map is a backdrop for a guessing task: there is no movement network, and its multiplayer modes are races rather than games of hidden information between opponents.

*Location-based* games such as Pokémon GO [4] take the opposite approach and use the player's physical position, so the city itself is the board. Paavilainen *et al.* [4] attribute Pokémon GO's mass appeal partly to this blending of play with everyday places, but also note the drawbacks: play depends on where a player lives and how far they can travel, and strategic interaction between players is limited.

This project sits between the two groups. It keeps the strategy of the board game and the recognisability of a real city, but moves the players' tokens rather than the players. That avoids the safety, accessibility and location problems of physical play while keeping the appeal of a familiar map.

### B. Existing Digital Versions

Three digital versions of the board game were reviewed:

- *SSY* [5] is a free website that enforces the rules. Mr X scans a QR code with his own phone to see his position and options, while the detectives enter their moves on a shared screen. It is played in one room around one screen, not over the internet.
- *Tabletop Simulator* [6] (NZ\$29 on Steam) is a physics sandbox in which players move pieces on a virtual copy of the board by hand. It can be played online, but it enforces no rules, so the game and Mr X's secret depend on the players being careful and honest.
- The *official PC adaptation* [7] (NZ\$22 on Steam, 2024) supports online play for up to six players on a redesigned London board, but had mostly negative reviews (23% positive of 21) at the time of writing.

All three use the fictional printed board, and none is a free online game that can be opened from a link.

### C. Cheating in Online Games

Making the game playable over the internet, one of this project's two themes, brings a problem the printed board game never has: keeping Mr X's position secret when every player's device is connected to the game. Research on cheating in online games is therefore relevant to the design. If the server sends Mr X's position to every player and the detectives' screens simply do not display it, a detective can still find it, for example by opening the browser's developer tools. Yan and Randell [8] list this, sending players information they are not supposed to see, as a common way of cheating in online games. The usual fix is to make the server the only part of the system that knows the full game state: players send the moves they want to make, the server decides what happens, and each player receives only what they are allowed to see. This was not a theoretical risk for this project. Earlier versions of the game had exactly this weakness: player identifiers that every client received also worked as credentials and players could request for Mrxs' location. The risk is greater because the project is open source. Anyone can read the repository to find which endpoints to call, and AI coding assistants make this easy even for players with little programming experience. A couple of user testers did exactly that, using AI to read the code and call the API directly to find Mr X. Secrecy therefore cannot depend on the code being hidden; it has to come from the server refusing to give out information. Section IV-D describes how the final version does that.

### D. Sending Live Updates to Browsers

In an online game, when one player moves, every other player's screen has to update straight away. Normally a browser only gets data when it asks for it, so there are three common ways to do this:

- *Polling*: the browser asks the server for the latest state every few seconds. This works everywhere, but updates are only as fast as the interval, and asking more often adds load on the server.
- *Server-Sent Events*: the browser keeps one connection open and the server sends updates down it as they happen. Messages can only go from the server to the browser.
- *WebSocket* [9]: a connection that stays open and carries messages both ways.

WebSocket on its own just carries messages. STOMP (Simple Text Oriented Messaging Protocol) adds named channels, called topics, that a browser can subscribe to, which suits a game well: one topic per game, and one private topic per player. Spring, the Java framework used for the server, supports STOMP directly. SockJS is a browser library that falls back to ordinary HTTP when a network blocks WebSockets. Section IV-A explains which of these was chosen and why.

### E. Map Tools

The game needs an interactive map of Wellington that can show the board's nodes and routes on top. Three map libraries were considered:

TABLE I  
COMPARISON WITH RELATED SYSTEMS.

| System                  | Real map   | Online     | Browser    | Rules           | Strategy    |
|-------------------------|------------|------------|------------|-----------------|-------------|
| Printed board game [1]  | No         | No         | No         | Players         | High        |
| SSY [5]                 | No         | No         | Yes        | Software        | High        |
| Tabletop Simulator [6]  | No         | Yes        | No         | Players         | High        |
| Official PC version [7] | No         | Yes        | No         | Software        | High        |
| GeoGuessr [3]           | Yes        | Yes        | Yes        | Software        | Low         |
| Pokémon GO [4]          | Yes        | Yes        | No         | Software        | Low         |
| <b>This project</b>     | <b>Yes</b> | <b>Yes</b> | <b>Yes</b> | <b>Software</b> | <b>High</b> |

- *Google Maps* is the best known, but it is a paid service: usage is billed per request and needs an API key [10].
- *Leaflet* is a free, open-source library. It draws the map using the web page's normal elements, which can slow down when many shapes are drawn.
- *MapLibre GL JS* [11] is also free and open source. It draws the map with the computer's graphics card (through WebGL), which keeps it smooth with many shapes on screen.

The free libraries need map data from somewhere. OpenStreetMap (OSM) is a free, community-made map of the world whose road data can be used under an open licence [12]. Overpass Turbo is a tool for downloading part of it, such as all the roads in Wellington, and CARTO provides free map styles built from it.

The game also needs each route between two nodes to follow real roads. Routing services such as Google's can work out a road path between two points, but each request costs money and takes time [10]. Using the free tools instead, the board can be built from real map data without paying per request. Sections IV-E and VI-A explain how the tools were chosen.

#### F. Measuring Usability

The System Usability Scale [13] is a 10-item questionnaire scored from 0 to 100. It is short, technology-agnostic and reliable with small samples. Across hundreds of studies the mean score is about 68 [14].

#### G. Summary of the Gap

No system found combines the four properties this project targets: the strategic depth of a hidden-movement board game, a real city's transport network as the board, real-time play in an ordinary browser opened from a link, and a server that keeps the hidden player's position secret. Table I summarises the comparison.

### III. REQUIREMENTS AND CONSTRAINTS

#### A. Functional Requirements

The rules follow the board game [1], adapted to Wellington:

- F1 *Lobby*. A host creates a game for two to six players and shares it by a six-character code, a link or a QR code. The host can remove players and start the game.

- F2 *Roles*. At the start, one player is chosen at random as Mr X and the rest become detectives; everyone starts on a different random node.
- F3 *Movement*. Each move follows one edge and spends a ticket for a transport mode on it: e-scooter, bus, train or ferry. Each detective starts with 12, 8, 6 and 2 of these; Mr X has unlimited ordinary tickets. Mr X may not move onto a detective. A detective with no legal move is skipped.
- F4 *Hidden information*. Detectives see Mr X's position only after his move in rounds 2, 8, 13, 18 and 24. Everyone always sees which ticket he used.
- F5 *Special tickets*. Mr X holds one Invisible ticket per detective, which crosses any edge and hides the mode, and two Double tickets, each giving two moves in one turn.
- F6 *Ending*. Detectives win by landing on Mr X or by leaving him no legal move; Mr X wins by surviving round 24. A game ends early if a player leaves or the current player does not move for 15 minutes.
- F7 *Real map*. The board is a graph of real Wellington locations, with edges drawn along real streets and lines and coloured by mode.

#### B. Non-Functional Requirements

- N1 *Standard hardware*. The game runs on an ordinary desktop or laptop in the current release of each of the three browser engines: Blink (Chrome, Edge), Gecko (Firefox) and WebKit (Safari). It needs only WebGL for the map and a WebSocket, or failing that HTTP streaming. No installation, plug-in or account is needed.
- N2 *Resilience*. A dropped connection must not end a game, and a client must catch up within seconds of reconnecting.
- N3 *Usability*. A SUS score above 68 [14].

#### C. Constraints

- C1 No paid per-request services such as metered routing or map APIs.
- C2 Self-hosted on the author's existing home server, in a single container, with no database.
- C3 One developer over 24 weeks, with a budget of \$500.
- C4 The original game's name and branding must not be used.

#### D. Scope

*In scope*: desktop browser play among friends who already know each other and talk by voice or in person; sharing a game by link or QR code; starting the whole server from one container image; and everything in F1–F7. *Out of scope*: user accounts and saved games; a mobile-first layout; computer-controlled opponents; in-game chat; and any use of players' real locations. There are two audiences. Friends who have agreed to play want a 30–60 minute game that is clear and fair. Open Day visitors try the game at a stand where a

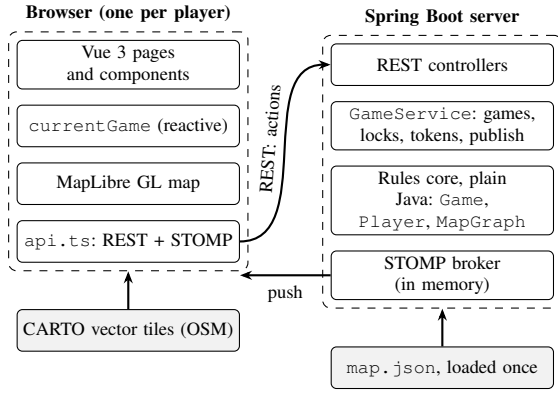


Fig. 1. System architecture. Actions travel as REST calls; state travels back as STOMP messages, one view per player. The rules core has no web-framework dependency.

demonstrator introduces it, and sets up the lobby, so the interface does not have to win a stranger’s attention in the first 30 seconds on its own; it has to let a newcomer join in seconds and understand a move after one explanation.

#### IV. DESIGN

##### A. Architecture

The system is a client–server design with an *authoritative* server (Fig. 1). The server holds every game in memory, applies every rule, and decides what each player may see. Browsers send intentions (“move to node 42 by bus”) as Representational State Transfer (REST) calls and receive state on STOMP topics; they never compute an outcome themselves.

The alternative considered was a *host-authoritative* design, in which the host’s browser runs the rules and the other players connect to it. It was rejected for two reasons. The host’s browser would hold the full game state, including Mr X’s position, so whenever the host played as a detective the secret would be on their device, breaking goal 2. And the game would end for everyone if the host closed the tab or their laptop went to sleep. A dedicated server avoids both problems, and it suits C2 because the home server already existed.

*How browsers and the server talk.* Communication is split into two directions:

- *Player actions* (create, join, start, move, leave) are ordinary web requests (REST). The main reason is error handling: when a move is illegal or it is not that player’s turn, the server can simply reply with an error code and a message, which the browser shows to the player. Ordinary requests are also easy to test.
- *Updates* Updates travel the other way over STOMP on a Web-Socket. Each browser subscribes to its game’s topic and its own private topic, and the server pushes the new state as soon as anything changes. Browsers never send anything on this connection; they only listen.

The other options from Section II were rejected. Polling alone would make every move take up to one polling interval to appear on other screens (6 s at the rate used here). Server-Sent Events could push updates just as quickly, but STOMP’s topics already give one channel per game and per player without extra

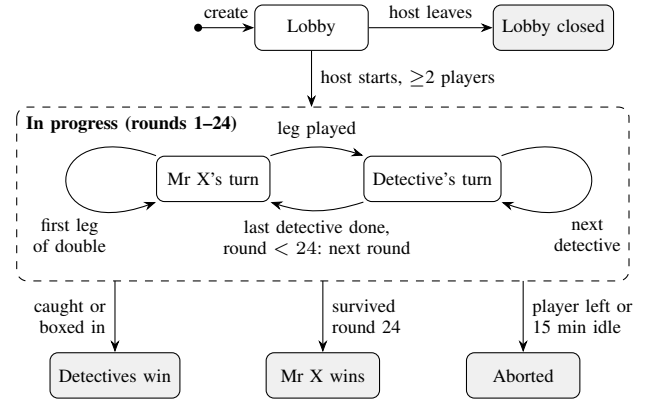


Fig. 2. Game state machine as implemented. Entering Mr X’s turn first checks that he can move; if detectives hold every neighbouring node, they win. Detectives with no legal move are skipped. A lost connection is not a transition.

code, and SockJS lets the connection fall back to ordinary HTTP on networks that block WebSockets (N1). Polling is still used as a backup in case a pushed update is missed (Section V-D).

*No database.* A game lasts under an hour and only exists while its players are online, so all games live in a concurrent hash map. A server restart ends every game; this is accepted for this version (C2) and listed as future work.

*Two languages, one contract.* Java was chosen for the server over Node.js because Spring provides a STOMP broker out of the box, and Java has mature coverage (JaCoCo) and mutation (PIT) tools for the test-quality measures promised in the proposal. The cost is that client and server cannot share types. The design answers this with one written contract: an OpenAPI 3.1 file documenting every REST endpoint and every STOMP topic, which must change in the same commit as any change to either side (Section V-G).

##### B. Game State Machine

Every operation is checked against the game’s state, so an out-of-order request is rejected rather than half-applied. Fig. 2 shows the state machine as implemented. It was revised during the project to match the code: an early “paused on disconnect” state was removed (Section V-D); “boxed in” was added as a detective win after a review found that games froze in that position; and “round end” became a transition rather than a state, because nothing waits there.

Two rules cover players who stop playing; neither comes from the board game, which assumes everyone stays at the table.

*Leaving ends the game.* If a player presses Leave during a game, the game ends for everyone with a message naming who left. Continuing without them was rejected. Without Mr X there is no game, and without a detective the game is no longer balanced, because Mr X’s number of Invisible tickets is set by the number of detectives at the start. Handing the missing player’s role to someone else would not work either. Ending the game is also the simplest rule to make correct. Losing a connection or refreshing the page is not leaving: the

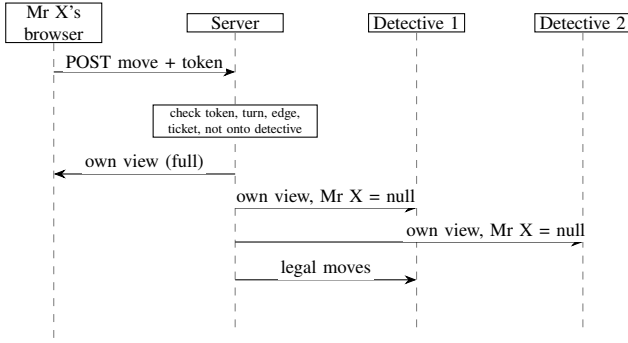


Fig. 3. One move by Mr X. The server validates the request, then sends each player a separately built view; outside a reveal round only Mr X's own view contains his node. Private topics are named by each player's secret token.

player keeps their place and catches up when they reconnect (Section V-D).

*Idle games are ended after 15 minutes.* If the player whose turn it is makes no move for 15 minutes, the game ends. Without this, a closed or forgotten tab would leave the other players waiting indefinitely, and the server would keep the game in memory forever. Shorter limits, or skipping a slow player's turn automatically, were rejected because players among friends often discuss a move for several minutes; 15 minutes is far longer than any turn in play-testing, so it only catches abandoned games.

### C. Domain Model

All rules live in three plain Java classes with no framework dependency: `Game` (lobby, start, moves, turns, reveals, endings, leaving, the idle abort and per-viewer views), `Player` (identity, role, position and tickets) and `MapGraph` (the board as sets of transport modes between adjacent nodes). A thin `GameService` keeps the live games, locks a game for every call, resolves player tokens and publishes new state. This split came from a mid-project rewrite. Originally the rules were spread through the Spring service and a lobby/Mr X/detective class hierarchy, which made them hard to test without mocking the graph. With the rules in a plain class, a test builds a game on a small hand-drawn map with exact start positions and checks one rule at a time. Unlimited tickets are stored as  $-1$ , so Mr X and detectives share one ticket model, and the random source and clock are injected so a test can fix the roles or jump 15 minutes ahead.

### D. Hidden Information and Role-Filtered State

Goal 2 is achieved by never building a message that contains Mr X's position for anyone but him. `Game.viewFor(viewer)` builds a separate snapshot for each viewer: Mr X sees everything, and anyone else sees his node only after he has finished a move in a reveal round. Mr X's move log carries his node only on a reveal round's final leg, the same for every viewer. After every change, the server publishes the public view to the game's topic, each player's own view to a private topic, and the legal moves to the player whose turn it is (Fig. 3).

Filtering the payload is necessary but not sufficient: a detective must also be unable to *obtain* Mr X's view. A late-project review found that the first implementation failed this in several ways. Player identifiers, which every client receives, doubled as credentials, so any player could request Mr X's view. The in-memory STOMP broker honoured wildcard subscriptions such as `/topic/games/{id}/players/**`, delivering every private view to an eavesdropper. It also relayed messages that clients sent to a topic as if the server had sent them. The redesign separates identity from authority:

- Each player receives a random secret *token* only in the response to their own create or join call. Every action carries it in an `X-Player-Token` header; the public identifier grants nothing.
- Private topics are named by the token, and the broker matches destinations exactly, so a wildcard subscription receives nothing.
- The server drops every message a client tries to send over STOMP, so clients can only listen.

Accounts and logins were considered and rejected: they add friction to a casual game among friends and protect nothing a per-game secret does not.

### E. Map Data Design

The board is a static JSON file of nodes, with latitude and longitude, and undirected edges, each listing its transport modes and a pre-computed line of coordinates along the real road or rail line. The server reads it once at start-up, validates it (an unknown mode or a dangling edge stops the server) and serves it to every client.

Live routing was the first idea and was rejected on cost and speed. Each edge needs a road path between two nodes. With about 400 edges and a directions service billed per thousand requests [10], every board load, or a cache to avoid repeating it, would add cost (C1); and an early prototype using a routing plug-in for Leaflet became unresponsive with a few hundred routes. The network does not change during a game, so paths are computed once, offline, from OSM road data, and stored with the graph. At run time the cost is one file download.

The graph's shape was designed for play rather than as an exact copy of the transport network (Fig. 4). Five clusters cover Wellington city, the Hutt Valley and the northern suburbs. Within each, nodes are joined by e-scooter and bus edges along streets, so most moves are short and local. A few train edges join the clusters quickly, and a ferry crosses the harbour. This mirrors the board game, where plentiful short moves are mixed with scarce long jumps, and the detectives' limited train and ferry tickets (6 and 2) make those jumps a resource to manage. Every edge is labelled with low-emission modes only, in keeping with Wellington's public transport and shared e-scooters. The train and ferry edges follow the real rail lines and harbour crossing, but bus and e-scooter edges simply follow Wellington roads; the map creator does not check them against real bus routes or e-scooter operating zones, so they represent what each mode could plausibly do rather than the actual network.

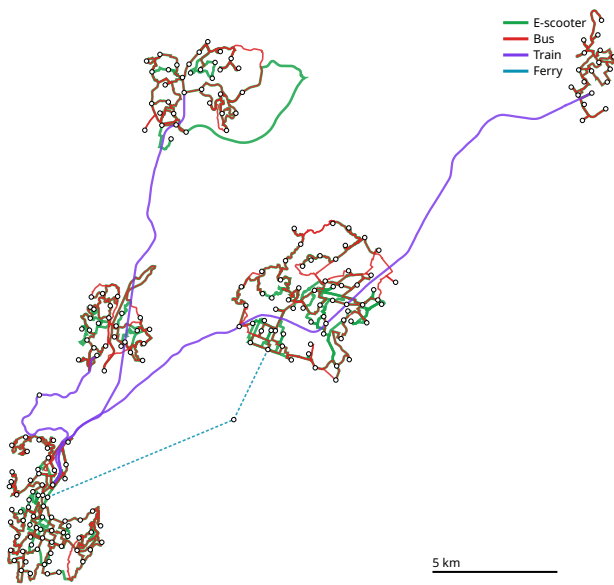


Fig. 4. The Wellington board: 216 nodes and 372 edges over about 22 km by 24 km. Edges follow their pre-computed paths and are coloured by mode; most carry both e-scooter and bus. Map data © OpenStreetMap contributors (ODbL).

#### F. User Interface Design

The interface has six screens: home, create, join, lobby, game board and game over. The lobby is the screen where players wait for the game to start. At the top it shows three ways for the host to invite others: the six-character join code, a link that opens the join page with the code already filled in, and a QR code of that link. The QR code was added with an Open Day stand in mind, so a visitor could scan it and join from their phone. In practice it is of little use: joining works, but the board was designed for a desktop screen, and on a phone the map and side panel do not fit well enough to play. The board (Fig. 5) splits the window between the map and a side panel. The map draws each edge in its mode's colour and each node as a small pie chart of the modes that stop there; players appear as rings in their own colour with a role symbol. The side panel lists players, the viewer's tickets, Mr X's ticket log with reveal rows, every reachable node with a button for each ticket that can pay for it, and the move to confirm. Clicking a reachable node on the map opens the same ticket buttons beside the node, so a move can be made without looking away from the map.

*Designing for committed players.* Both audiences in Section III arrive having already decided to play: friends have agreed on a game, and Open Day visitors have been introduced to it by a demonstrator. The interface therefore does not need to grab attention in the first 30 seconds, as an app competing in a store or a casual web game must, and this changed several design choices:

- The home page is a title and two buttons, Create and Join, with no splash screen, introductory animation, sound or reward loop. Effort went into the board, where players spend their time.
- The board is information-dense rather than minimal. The

side panel shows every player, every ticket count, the full Mr X log and every reachable node at once, because committed players use all of it to plan and a sparse first impression is not the goal.

- The full 24-round game was kept, with its 30–60 minute length, instead of being shortened into a quick session to keep a casual visitor hooked.
- The rules are not taught by the interface. A host or demonstrator explains them in person, so the screen can assume a player who knows what a reveal round is. The cost is that new players asked for a tutorial (Section VI-C), which remains future work.
- There is no automatic skip of a slow turn: committed players are expected to think, and only a 15-minute idle limit protects against an abandoned tab.

Several of these features came from user testing (Section VI-C): the transport-mode popups, the taller move list, the centred turn indicator and the reveal rows in the log. A dark basemap was chosen so the four mode colours stand out; a light and a “Voyager” style are available for personal preference.

#### G. Real-World Issues

*Sustainable transport.* The game's world contains only e-scooters, buses, trains and a ferry. Players spend 24 rounds planning journeys across the region on those modes alone, along real streets and the real rail lines and ferry crossing. No claim is made that playing changes how anyone travels.

*Sustainable software.* The same priority applied to every infrastructure decision. Pre-computing routes replaces hundreds of routing requests per game with one static file. The server is a single process in a 231 MB container image on hardware the author already owned, with no database, no cloud services and no new devices. The whole stack is open source, so the game is not tied to a vendor's pricing or lifetime. The one remaining inefficiency, an uncompressed 1.86 MB map file, is discussed in Section VII.

*Safety and social impact.* Games like Pokémon GO require players to walk around real streets, which raises safety concerns such as traffic. In this game only the players' pieces move on the map, while the players stay at their computers, so it is safe to play and open to people who cannot easily travel, including those with limited mobility. The game is also social: detectives talk to each other to plan their moves. At the Open Day, it showed prospective students a working game built in an engineering degree.

*Privacy and ethics.* The game stores no accounts, email addresses or locations; a player's name lives in memory only until the game ends. User testing was voluntary, involved fellow students, and was covered by the approval the course coordinator gave at the proposal stage. Questionnaire responses were anonymised by removing the email column before analysis.

*Intellectual property.* The mechanics are used with Ravensburger's permission, the original name is not used (C4), the code is released under the GNU General Public Licence version 3, and the map shows the attributions that OSM and CARTO require [12].

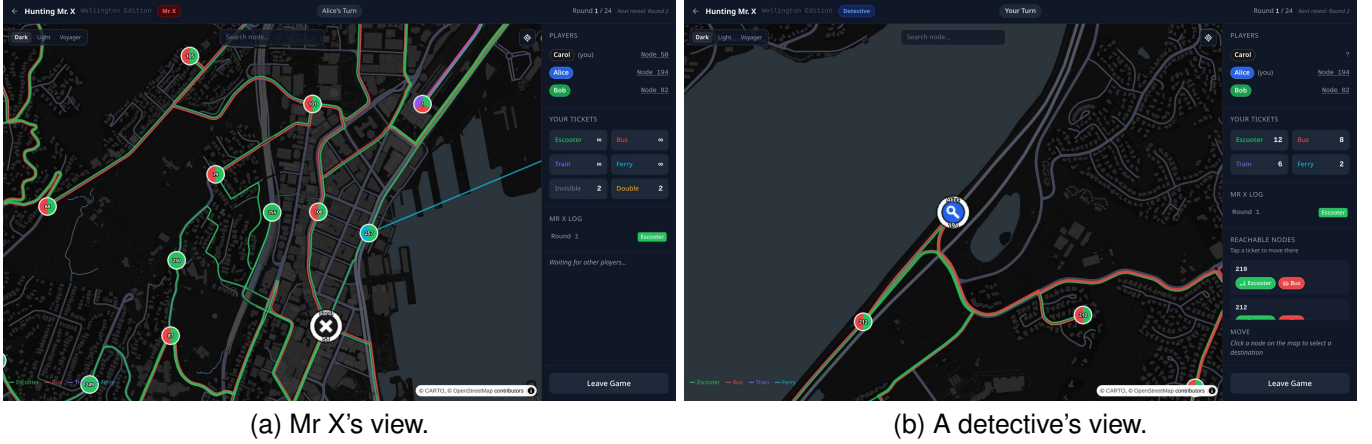


Fig. 5. The game board in round 1, after Mr X has moved by e-scooter. (a) Mr X sees his own marker, his unlimited ordinary tickets and his Invisible and Double tickets. (b) In a detective's view of the same game, Mr X's node shows as "?", and each reachable node offers a button per ticket that can pay for it. Both players receive the same log entry, but only Mr X's view contains his position.

TABLE II  
TECHNOLOGY STACK.

| Component                              | Reason                                                                                                                           |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Java 21, Spring Boot 4.0               | Built-in STOMP broker; records for immutable views; mature test tooling                                                          |
| Vue 3, TypeScript, Vite                | Single-file components with less boilerplate than React for a small solo project; types catch message-format drift at build time |
| Tailwind CSS 4                         | Consistent dark and light themes without a component library                                                                     |
| MapLibre GL JS 4.7                     | WebGL vector rendering, open source, no key (Section VI-A)                                                                       |
| @stomp/stompjs, SockJS                 | Automatic reconnect; HTTP fallback where WebSockets are blocked                                                                  |
| CARTO basemaps, OSM data               | Free, label-free vector styles; open road data                                                                                   |
| JUnit 5, Mockito, AssertJ, JaCoCo, PIT | Unit to integration testing, coverage and mutation testing                                                                       |
| Docker, Cloudflare Tunnel              | One reproducible image; HTTPS to a home server without opening ports                                                             |

## V. IMPLEMENTATION

### A. Technology Choices

Table II lists the components used and why each was chosen.

### B. Server

The backend is about 770 lines of Java code (1,460 including documentation) in 22 files.

When a player submits a move, `Game.move` checks, in order, that:

- 1) the game is in progress and it is this player's turn;
- 2) the ticket is valid (the first half of a double move is sent as, for example, `DOUBLE_BUS`, and a second double cannot be started during a double);
- 3) the two nodes are connected;
- 4) the ticket matches a transport mode on that connection, unless it is an Invisible ticket;
- 5) the player still has that ticket;
- 6) Mr X is not moving onto a detective.

Only if every check passes are the ticket spent and the player moved, so a rejected move changes nothing. The turn then passes on. After Mr X moves, it goes to the first detective who can move; after a detective, to the next detective, or to a new round once all have moved. Before each of Mr X's turns, the server checks that he has a legal move; if he does not, the detectives win.

Several players can send requests to the same game at the same moment, for example two players joining at once. To stop these requests interfering with each other, each game has its own lock: the server handles one request for a game at a time, while different games run in parallel. Making requests within a game wait for each other costs little, because a game only receives a few requests per second.

Every 30 s, the server checks for games where the current player has not moved within the idle limit and ends them. Settings such as the detectives' ticket counts, the idle limit and which map file to load are all kept in one configuration file.

### C. Client

The frontend is 23 source files and about 2,400 lines. Game state is one `Vue reactive()` object that every page and component reads; a state-management library (Pinia) was used at first and removed, because two small stores did not justify a dependency. All server traffic goes through one module, `api.ts`: a single helper for REST calls, and `keepUpToDate()`, which opens the STOMP connection and maps each topic to a handler. The map component is the largest file, at about 700 lines. It loads the graph from the server, adds it as GeoJSON sources, draws one line layer per mode with small offsets so parallel modes sit side by side, and pre-draws the 16 possible pie icons for node mode combinations.

### D. Resilience to Dropped Connections

The original design paused a game when a player disconnected and resumed it within a 60 s grace period. User testing on university and home Wi-Fi showed WebSockets

dropping far more often than expected, so pausing would have interrupted games constantly. The design changed so that a dropped connection is not a game event at all. The STOMP client reconnects automatically, re-subscribes, and immediately fetches the full state over REST; a REST refresh also runs every 6 s regardless. Because every message is a complete snapshot rather than a change, a missed message costs at most one refresh interval and can never leave a client inconsistent. This is why the server sends whole views rather than differences, at the price of slightly larger messages.

### E. Building the Wellington Map

Building the map was the hardest part of the project, and the rest of the game could not be fully tested until it was done. Road data for the region was exported from OSM with Overpass Turbo (about 5.7 MB of GeoJSON), and train lines were traced separately. A first approach that laid OSM road segments under each edge produced loops and doubled lines where segments overlapped. Fully automatic generation tended to connect every node to every neighbour, and some routed paths between node pairs wandered far from the direct line.

The solution was a dedicated browser tool, the *map creator*, built on MapLibre. It snaps nodes to the road network, routes each edge along roads with Dijkstra's shortest-path algorithm, and generates the network in layers: train stations and lines first, then bus clusters around each area, then e-scooter edges limited to short distances (at most 900 m). A road-use registry stops two edges claiming the same street, which removed the doubled lines. Generation is followed by manual editing (adding, joining, re-moding and renumbering nodes, with undo), a connected-components view to make sure every node is reachable, and export. The tool was later refactored from one 2,500-line file into nine modules of about 1,500 lines in total, and its output was checked byte for byte against the original before any behaviour changed.

### F. Deployment

The Docker build compiles the frontend, copies it into the server's static folder and packages a single Java archive; the running container is one Java process that serves the pages, the REST API and the WebSocket endpoint. An earlier image ran nginx beside the server under a process supervisor, with a generated nginx configuration of over 100 lines. Removing it took away a process and a configuration language without losing anything, since Spring can serve static files under the same base path. A deploy script rebuilds from a Git tag, swaps the container and deletes old images, and a Cloudflare Tunnel publishes the server over HTTPS without opening ports on the home network.

### G. Development Process and Use of AI

Development ran in short iterations, each ending in something runnable: planning and the map-library benchmark (March–May), the lobby (May), game logic and map rendering (June), the map creator and Wellington graph (June–August),

Docker deployment (July–August), user testing (August–September), and a review, rewrite and refactoring phase (September). The repository holds 136 commits.

*AI-assisted development.* Claude Code, a command-line AI coding assistant, was used throughout. It drafted code and tests from written specifications, reviewed code for bugs, carried out refactorings, and kept documentation and diagrams consistent with the code. It was most useful for repetitive changes across many files, such as renaming something in 20 places, and for reviewing code. An AI-assisted code review found two of the most serious bugs: the security weakness that let players see Mr X's position (Section IV-D), and a bug where the game froze if Mr X had no legal move. Its weaknesses were just as clear. It produced plausible but wrong documentation (for example, describing a pause-on-disconnect feature that had been removed), it tended to add layers and abstractions that were not needed, and early tests it wrote often checked getters rather than behaviour, which a coverage figure alone would not reveal. Not every AI-generated change was reviewed line by line, so the project relied on automated checks to catch mistakes: the automated test suite; mutation testing, which showed whether the tests really checked behaviour (Section VI-B); and a local pre-commit hook that rejected any commit changing the API without also updating the OpenAPI file, so the frontend and backend could not silently fall out of step.

*Tool critique.* Spring's simple broker made STOMP quick to adopt, but its permissive defaults (wildcard subscriptions, client publishing to topics) caused the most serious bug: a framework's convenient defaults are not necessarily secure ones. MapLibre performed well, but version 4.7 ignored the first resize of its container, leaving a half-blank map in tabs that opened in the background; a resize observer fixed it. SockJS remains useful on restrictive networks, but its client is no longer actively developed and could be replaced by STOMP over a plain WebSocket if the fallback proves unnecessary. Selenium, used first for an end-to-end test, acted only as an HTTP client and was flaky; a browserless test with real HTTP and STOMP clients replaced it, and Playwright was used for scripted browser checks instead.

## VI. EVALUATION

### A. Map Library Benchmark

Before any map code was written, three libraries were compared in a harness that loads the same scenario in each and records the time until the library reports that rendering has finished. A Python script drove Firefox through Selenium, served every page from a local server so network effects were equal, made one unrecorded warm-up load, and recorded 10 runs per library per scenario. The *baseline* scenario had four Wellington nodes; the *stress* scenario had 200 nodes sampled from real road endpoints and 285 edges, the expected size of the final board. Fig. 6 shows the results.

Leaflet was fastest in both scenarios, by 0.17 s at baseline and 0.10 s under stress; Google Maps was slowest and needs a billed key. All three rendered the stress scenario in under 3 s. MapLibre was chosen over the slightly faster Leaflet for

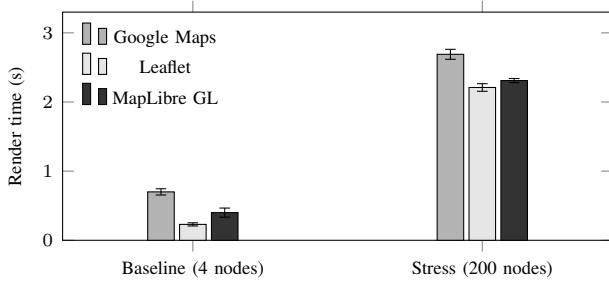


Fig. 6. Mean render time over 10 runs per library, with one standard deviation as error bars (Firefox, Selenium, local server). Means for the stress scenario: Google Maps 2.69 s, Leaflet 2.21 s, MapLibre GL 2.31 s.

TABLE III  
AUTOMATED TEST SUITE.

| Layer          | Tests | What it checks                                                           |
|----------------|-------|--------------------------------------------------------------------------|
| Unit           | 82    | Every rule in Game, Player, MapGraph                                     |
| Mock           | 17    | Locking, tokens, what is published where                                 |
| Lifecycle      | 8     | Scripted complete games through the service                              |
| Integration    | 20    | HTTP status codes and errors; STOMP filtering; wildcard and SEND attacks |
| Functional     | 4     | Full games over real HTTP and STOMP                                      |
| Property, fuzz | 4     | 300 seeded random games; random requests to every endpoint               |

three reasons. It was the most consistent under load (standard deviation 0.030 s against Leaflet’s 0.055 s). It draws vector tiles and data layers on the graphics processor, so styling the graph by data (colour per mode, offsets, pie icons) is a layer property rather than thousands of DOM elements. And it is fully open source with no key (C1). The benchmark has clear limits: one browser on one machine, 10 runs, and synthetic scenarios rather than the final board.

### B. Functional Correctness

The backend has 135 automated tests in six layers (Table III), all passing on the final version, and none needing a browser. Unit tests build games on hand-drawn maps of five to eight nodes with fixed start positions, so each rule is tested against an exact expected outcome: for example, that both legs of a double move happen before any detective moves, that a reveal shows only the second leg’s node, or that a detective with no legal move is skipped without spending a ticket. Integration tests run the real server and talk to it over HTTP and STOMP. Functional tests play complete games with one HTTP and one STOMP client per player, checking the game’s invariants after every move.

*Regression tests.* The late-project review found eight bugs, three of them high severity: the game froze when Mr X was boxed in (which also made an earlier end-to-end test fail at random, because 30% of four-player starts on the small test map box him in on turn 1); any player could see Mr X’s position or act as another player (Section IV-D); and a detective leaving froze the game or skipped a turn. Each fix has at least one test that failed before it. The two STOMP attacks were written as tests and run against the old configuration,

TABLE IV  
PLAY-TEST FINDINGS AND RESULTING CHANGES.

| Round | Finding                              | Change                                   |
|-------|--------------------------------------|------------------------------------------|
| 1     | Long names broke the layout          | 20-character limit on client and server  |
| 1     | Easy to zoom away from the region    | Zoom limited to Wellington               |
| 1     | Move list too short to use           | Taller, viewport-relative panels         |
| 1     | Hard to tell a node’s modes          | Pie-chart nodes; popup on click          |
| 1     | Final state not visible at the end   | Game-over summary screen                 |
| 1     | Modes hard to tell apart             | More distinct mode colours               |
| 1     | Stale highlight after a second click | Selection reset fixed                    |
| 2     | Whose turn is it?                    | Turn badge centred in header             |
| 2     | When is the next reveal?             | “Next reveal: Round <i>n</i> ” in header |
| 2     | Reveals lost in the log              | Reveal rows between log entries          |
| 2     | “R12” unclear                        | “Round 12” written in full               |

where the eavesdropper did receive Mr X’s private view, before the fix made them pass.

*Testing the tests.* Coverage shows what tests execute, not what they check, so three measures were used:

- *Coverage* (JaCoCo): 99.2% of lines and 97.8% of branches across the backend, up from 65% and 45% before the rewrite, when the rules engine was mocked out of most tests.
- *Mutation testing* with PIT introduces small faults, such as flipping a comparison or deleting a call, and checks that some test fails. All 199 mutants generated in the rules and service code were killed.
- *Property-based and fuzz testing*: 300 seeded random games on the real map, each checked after every move for invariants such as “Mr X’s node never appears in a detective’s view outside a reveal”, “no ticket count goes negative” and “every game ends”; and random JSON sent to every endpoint, which must never cause a server error.

The frontend has no automated tests. It was manually tested after each update.

*Browser support (N1).* The game was developed and play-tested in Firefox and Chromium-based browsers. Safari was not tested systematically as the author does not have access to an apple device.

### C. User Evaluation

Two rounds of play-testing with fellow students shaped the interface, and players also completed the SUS questionnaire.

*Iterative rounds.* After each round the observed problems were listed and fixed before the next (Table IV). Round 1 exposed basic usability gaps; round 2 found only presentation issues, suggesting the first round’s fixes worked.

*SUS results.* Seven players (six detectives and one Mr X, in games of four to six players) completed the questionnaire after a full game. The overall mean was 72.5 (range 60–95): 72.9 for detectives ( $n = 6$ ) and 70.0 for Mr X ( $n = 1$ ). This is above the average of 68 [14], meeting N3. In free-text comments, three of the seven asked for a tutorial or rules walk-through; others found the Mr X log hard to read at first or the reachable-node list cramped on small screens, praised

TABLE V  
REQUIREMENTS AND OUTCOMES.

| Req.       | Evidence                                                     | Status |
|------------|--------------------------------------------------------------|--------|
| F1–F6      | 135 tests including unit tests per rule; 100% mutation score | Met    |
| F4, goal 2 | Per-viewer views, tokens, exact-match broker; attack tests   | Met    |
| F7         | 216-node, 372-edge graph on real streets (Fig. 4)            | Met    |
| N1         | Chrome and Firefox used throughout; Safari untested          | Partly |
| N2         | Reconnect and REST re-sync; 6 s worst case                   | Met    |
| N3         | SUS 72.5 against 68 ( $n = 7$ )                              | Met    |
| C1–C4      | No paid services; one container; within budget               | Met    |

the game as fun and replayable, and asked for custom lobby rules.

*Limitations.* Seven participants is too few for a precise score: one respondent can move the mean by several points. Only one participant played Mr X; the sample is a convenience sample of engineering students, likely more at ease with new software than the general public; and the questionnaire was given after the round-2 fixes, so there is no before-and-after comparison.

#### D. Requirements Traceability

Table V summarises each requirement against its evidence.

### VII. DISCUSSION

*What worked.* Three decisions carried the project. Pre-computing the map turned the most expensive and least predictable part of the design, routing, into a one-off offline task with no per-request cost. Putting the rules in a plain class, separate from the framework, made exhaustive and exact tests possible; the rise from 65% to 99% coverage came mostly from that change rather than from writing more tests. Sending complete, per-viewer snapshots made both secrecy and resilience simple: there is no client-side game logic that could leak, and no sequence of updates that a missed message could corrupt.

*What was hard.* Building a playable map took far longer than any code. Real road networks are messy, and a graph that is fun to play needs judgement about density, connectivity and the balance between modes that no generator captured on its own. Connection drops on real networks also forced a late design change; the lesson is to treat connectivity as unreliable from the start rather than as an exception.

*Security lesson.* The most serious defect was not in the game rules but in configuration: identifiers that doubled as credentials, and a message broker that by default allowed wildcard subscriptions and client publishing. Filtering the payload correctly was not enough, because the attack went around it. The existing tests did not catch it, because they only checked that the game worked when players used it as intended. It was found by testers and a code review that deliberately tried to get Mr X's position. When keeping a secret is a core requirement, the tests need to include attempts to break the system, not just normal use.

*Trade-offs revisited.* With no database, a restart ends every game; for short games among friends on a home server this has

not been a problem, but it rules out long or scheduled games. The map file is sent uncompressed at 1.86 MB because its edge paths hold about 28,000 coordinate pairs at full precision; HTTP compression and rounding to about one-metre precision would shrink it several-fold with no visible difference, saving data on every board load. The desktop-first layout suits the intended setting but rules out phone play, which also makes the lobby's QR code largely redundant.

*Threats to validity.* The Open Day demonstration was observed informally rather than measured; the benchmark used one browser on one machine; the user study is small and homogeneous; and the frontend's correctness rests on manual tests rather than an automated suite.

### VIII. CONCLUSIONS AND FUTURE WORK

This project built a browser-based, real-time hidden-movement game on a real map of Wellington. An authoritative Java server enforces every rule and sends each player a separately built view, so Mr X's position never reaches a detective's device outside a reveal round. A Vue and MapLibre client draws a pre-computed 216-node graph over open map data. All three goals were met: every rule is enforced and tested, with all 199 mutants killed; the secret holds against deliberate attacks on both the REST and STOMP interfaces; and players rated usability 72.5 on the SUS. The game is deployed and can be joined by anyone with a link, over the internet, from any desktop browser, and it has already been run successfully as an Open Day demonstration.

The finished system opens several projects for a future student:

- *Computer players and game balance.* Computer-controlled players would allow a single-player mode. They would also make it possible to check whether the game is fair: by having computer players play thousands of games against each other, a student could measure how often each side wins and how that changes with the number of tickets, the reveal rounds or the layout of the map. This is practical because the rules are in one self-contained, well-tested class that can run games without the server.
- *Boards for any city.* The map creator's layered generation could become a pipeline from open transport data to a playable board, with balance checked by those bots, so the game could be played on any city.
- *Distributed, persistent games.* Persistence and accounts would allow slow games played over days, and running games across several server instances would make the system a test bed for keeping distributed state consistent while preserving per-player secrecy.
- *Open Days and a more faithful map.* Building on the successful Open Day, a short tutorial (the most common user request), a mobile layout and a "spectator" view for a projector would make the game easier to run with the public, and a structured study at a future Open Day could measure visitors' experience. Building bus and e-scooter edges from real bus route data and e-scooter operating zones would make the board a faithful model of the

network; a study could then measure whether playing improves players' knowledge of it.

- *Onboarding and reach.* Configurable lobby rules and a larger user study with more Mr X players.

#### ACKNOWLEDGMENTS

The author thanks Jens Dietrich and Stuart Marshall for their supervision and feedback, the course coordinator for approving the user testing, Ravensburger for permission to use the game mechanics, and the students who play-tested the game.

#### REFERENCES

- [1] Projekt Team III, *Hidden-Movement Detective Board Game: Rules of Play*, Ravensburger, Ravensburg, Germany, 1983, winner of the Spiel des Jahres award, 1983.
- [2] M. Bao, "Hunting Mr. X: Wellington edition, source repository (tag ENGR\_489\_FINAL\_ARTEFACT)," [Online]. Available: [https://github.com/Ming-Bao/HuntingMrX/tree/ENGR\\_489\\_FINAL\\_ARTEFACT](https://github.com/Ming-Bao/HuntingMrX/tree/ENGR_489_FINAL_ARTEFACT), 2026.
- [3] GeoGuessr AB, "GeoGuessr," [Online]. Available: <https://www.geoguessr.com>, 2013, accessed: Oct. 2, 2026.
- [4] J. Paavilainen, H. Korhonen, K. Alha, J. Stenros, E. Koskinen, and F. Mäyrä, "The Pokémon GO experience: A location-based augmented reality mobile game goes mainstream," in *Proc. CHI Conf. Human Factors in Computing Systems*, Denver, CO, USA, 2017, pp. 2493–2498.
- [5] "SSY: A simple browser version of the board game," [Online]. Available: <https://ssy.pvx.sk>, accessed: Oct. 4, 2026.
- [6] Berserk Games, "Tabletop Simulator," Steam. [Online]. Available: <https://store.steampowered.com/app/286160/>, 2015, accessed: Oct. 4, 2026.
- [7] BitBarons GmbH and Markt+Technik Verlag GmbH, "Official PC adaptation of the 1983 board game," Steam. [Online]. Available: <https://store.steampowered.com/app/3073810/>, 2024, accessed: Oct. 4, 2026.
- [8] J. Yan and B. Randell, "A systematic classification of cheating in online games," in *Proc. 4th ACM SIGCOMM Workshop on Network and System Support for Games (NetGames)*, Hawthorne, NY, USA, 2005, pp. 1–9.
- [9] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Internet Engineering Task Force, Dec. 2011.
- [10] Google, "Google Maps Platform pricing," [Online]. Available: <https://mapsplatform.google.com/pricing/>, accessed: Oct. 2, 2026.
- [11] MapLibre, "MapLibre GL JS documentation," [Online]. Available: <https://maplibre.org/maplibre-gl-js/docs/>, accessed: Oct. 2, 2026.
- [12] OpenStreetMap Contributors, "OpenStreetMap copyright and licence (ODbL)," [Online]. Available: <https://www.openstreetmap.org/copyright>, accessed: Oct. 2, 2026.
- [13] J. Brooke, "SUS: A "quick and dirty" usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.
- [14] J. Sauro, *A Practical Guide to the System Usability Scale: Background, Benchmarks and Best Practices*. Denver, CO, USA: Measuring Usability LLC, 2011.